



《AI大模型Ragent项目》——Rerank重排序与辅助工具

来自： 拿个offer-开源&项目实战

 马丁

2026年04月15日 08:18

前七篇讲完了 Chat 子系统（六篇）和 Embedding 子系统（一篇），覆盖了 infra-ai 模块 9 个包中的 6 个：config（配置）、enums（枚举）、model（路由核心）、http（HTTP 基础设施）、chat（对话）、embedding（向量化）。这一篇讲剩下的 3 个包：rerank（重排序子系统）、token（Token 估算）、util（工具类），把整个 infra-ai 模块收尾。Rerank 是三种能力中的最后一个——和 Chat、Embedding 遵循同样的三层接口设计，复用同一套路由和熔断机制。但 Rerank 有两个独特之处：一是百炼的 Rerank 实现里有一套去重 + 回填的防御性逻辑，处理混合检索场景的边缘情况；二是 NoopRerankClient 用空对象模式实现了优雅降级——没有 Rerank 模型时系统不报错，而是回退到简单截断。Token 估算和响应清洗是两个小工具，代码量不大，但在 RAG 流程中经常用到。一并在本篇讲完。

Rerank 重排序子系统

1. Rerank 在 RAG 中的角色

回顾一下 RAG 的检索阶段。用户提问“AirPods Pro 2 的保修期是多久？”，系统先通过向量检索（Bi-Encoder）从 100 万个 Chunk 中快速召回 Top-20 候选。这 20 个候选覆盖面广（保证了召回率），但排序不一定准——向量相似度是粗略的语义匹配，和 query 的真正相关度之间有差距。Rerank 的作用是对这 20 个候选做精排。它用 Cross-Encoder 模型逐对评估 query 和每个候选的相关度，给出精确的分数，返回最相关的 Top-5 喂给大模型生成回答。两阶段策略的分工：粗排（向量检索）保证覆盖率——宁可多召回一些不太相关的，也不要漏掉真正相关的；精排（Rerank）保证准确率——从粗排结果中挑出真正最相关的。这个数据漏斗在第一篇的架构总览中提过：100 万文档 → 向量检索召回 20 → Rerank 精排 5 → 大模型生成 1 个答案。

2. 接口设计

2.1 RetrievedChunk 数据结构

Rerank 操作的数据单元是 RetrievedChunk——定义在 framework 模块，是跨层共享的数据结构：

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RetrievedChunk {

    /**
     * 命中记录的唯一标识（向量库主键）
     */
    private String id;

    /**
     * 命中的文本内容（Chunk 正文）
     */
    private String text;

    /**
     * 命中得分（数值越大相关性越高）
     */
    private Float score;
}
```

三个字段：id 是向量数据库中的主键，text 是 Chunk 的文本内容，score 是相关度分数。向量检索阶段，score 是向量相似度分数；Rerank 之后，score 会被更新为 Rerank 模型给出的 relevance_score。RetrievedChunk 放在 framework 模块而不是 infra-ai 模块——因为检索层、Rerank 层、生成层都用到它，是跨模块共享的约定。

2.2 RerankService 业务层接口

```
public interface RerankService {

    /**
     * 对候选文档进行精排，按相关度重新排序，返回前 topN 条
     *
     * @param query      用户问题
     * @param candidates 向量检索召回的候选文档
     * @param topN       最终保留的条数
     * @return 精排后的前 topN 条文档
     */
    List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates, int topN);
}
```

只有一个方法，语义很清晰：传入 query 和候选列表，返回精排后的 Top-N。和 LLMService、EmbeddingService 的设计理念一致——业务层只看到这个接口，不感知供应商、路由、熔断等 infra 细节。

2.3 RerankClient 供应商接口

```
public interface RerankClient {

    String provider();

    List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates,
                                int topN, ModelTarget target);
}
```

多了一个 ModelTarget 参数——和 ChatClient、EmbeddingClient 同样的设计。provider() 返回供应商标识，路由服务通过它查找对应的客户端实例。

3. BaiLianRerankClient 完整代码

```
@Service
@Slf4j
@RequiredArgsConstructor
public class BaiLianRerankClient implements RerankClient {

    private final OkHttpClient httpClient;

    @Override
    public String provider() {
        return ModelProvider.BAI_LIAN.getId();
    }

    @Override
    public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates,
                                       int topN, ModelTarget target) {
        if (candidates == null || candidates.isEmpty()) {
            return List.of();
        }

        List<RetrievedChunk> dedup = new ArrayList<>(candidates.size());
        Set<String> seen = new HashSet<>();
        for (RetrievedChunk rc : candidates) {
            if (seen.add(rc.getId())) {
                dedup.add(rc);
            }
        }

        if (topN <= 0 || dedup.size() <= topN) {
            return dedup;
        }
    }
}
```

```
        return doRerank(query, dedup, topN, target);
    }

    private List<RetrievedChunk> doRerank(String query, List<RetrievedChunk> candidates,
                                          int topN, ModelTarget target) {
        AIModelProperties.ProviderConfig provider =
            HttpResponseHelper.requireProvider(target, provider());

        if (candidates == null || candidates.isEmpty() || topN <= 0) {
            return List.of();
        }

        JsonObject reqBody = new JsonObject();
        reqBody.addProperty("model", HttpResponseHelper.requireModel(target, provider()));

        JsonObject input = new JsonObject();
        input.addProperty("query", query);

        JsonArray documentsArray = new JsonArray();
        for (RetrievedChunk each : candidates) {
            documentsArray.add(each.getText() == null ? "" : each.getText());
        }
        input.add("documents", documentsArray);

        JsonObject parameters = new JsonObject();
        parameters.addProperty("top_n", topN);
        parameters.addProperty("return_documents", true);

        reqBody.add("input", input);
        reqBody.add("parameters", parameters);

        Request request = new Request.Builder()
            .url(ModelUrlResolver.resolveUrl(
                provider, target.candidate(), ModelCapability.RERANK))
            .post(RequestBody.create(reqBody.toString(), HttpMediaTypes.JSON))
            .addHeader("Authorization", "Bearer " + provider.getApiKey())
            .build();

        JsonObject respJson;
        try (Response response = httpClient.newCall(request).execute()) {
            if (!response.isSuccessful()) {
                String body = HttpResponseHelper.readBody(response.body());
                log.warn("{} rerank 请求失败: status={}, body={}",
                    provider(), response.code(), body);
                throw new ModelClientException(
                    provider() + " rerank 请求失败: HTTP " + response.code(),
                    ModelClientErrorType.fromHttpStatus(response.code()),
                    response.code()
                );
            }
            respJson = HttpResponseHelper.parseJson(response.body(), provider());
        } catch (IOException e) {
            throw new ModelClientException(
                provider() + " rerank 请求失败: " + e.getMessage(),
                ModelClientErrorType.NETWORK_ERROR, null, e);
        }

        JsonObject output = requireOutput(respJson);

        JsonArray results = output.getAsJsonArray("results");
        if (CollUtil.isEmpty(results)) {
            throw new ModelClientException(
                provider() + " rerank results 为空",

```

```
        ModelClientErrorType.INVALID_RESPONSE, null);
    }

    List<RetrievedChunk> reranked = new ArrayList<>();
    Set<String> addedIds = new HashSet<>();

    for (JsonElement elem : results) {
        if (!elem.isJsonObject()) {
            continue;
        }
        JsonObject item = elem.getAsJsonObject();

        if (!item.has("index")) {
            continue;
        }
        int idx = item.get("index").getAsInt();

        if (idx < 0 || idx >= candidates.size()) {
            continue;
        }

        RetrievedChunk src = candidates.get(idx);

        Float score = null;
        if (item.has("relevance_score") && !item.get("relevance_score").isJsonNull()) {
            score = item.get("relevance_score").getAsFloat();
        }

        RetrievedChunk hit = score != null
            ? new RetrievedChunk(src.getId(), src.getText(), score) : src;
        reranked.add(hit);
        addedIds.add(src.getId());

        if (reranked.size() >= topN) {
            break;
        }
    }

    if (reranked.size() < topN) {
        for (RetrievedChunk c : candidates) {
            if (addedIds.add(c.getId())) {
                reranked.add(c);
            }
            if (reranked.size() >= topN) {
                break;
            }
        }
    }

    return reranked;
}

private JsonObject requireOutput(JsonObject respJson) {
    if (respJson == null || !respJson.has("output")) {
        throw new ModelClientException(
            provider() + " rerank 响应缺少 output",
            ModelClientErrorType.INVALID_RESPONSE, null);
    }
    JsonObject output = respJson.getAsJsonObject("output");
    if (output == null || !output.has("results")) {
        throw new ModelClientException(
            provider() + " rerank 响应缺少 results",
            ModelClientErrorType.INVALID_RESPONSE, null);
    }
}
```

```
        return output;
    }
}
```

代码不短，但逻辑分层清晰：rerank 方法做前置处理（去重 + 短路），doRerank 做实际的 HTTP 调用和响应解析。逐段拆解。

4. 去重：混合检索的重复问题

rerank 方法入口处有一段去重逻辑：

```
List<RetrievedChunk> dedup = new ArrayList<>(candidates.size());
Set<String> seen = new HashSet<>();
for (RetrievedChunk rc : candidates) {
    if (seen.add(rc.getId())) {
        dedup.add(rc);
    }
}
```

为什么需要去重？

Ragent 的检索阶段采用混合检索——向量检索和 BM25 关键词检索并行执行，结果合并后做 RRF 排名融合。同一个 Chunk 可能同时被向量检索和 BM25 命中。合并后的候选列表中，这个 Chunk 就出现了两次。

当然，这里其实是为了兜底，从 Ragent AI 角度出发，会在搜索后的后置处理器里优先去重。

如果不去重就传给 Rerank API，documents 数组里有两条完全一样的文本，白白浪费 Token（Rerank 模型按 Token 计费），而且返回的 results 中可能出现两条指向同一个 Chunk 的结果，占掉 topN 中的两个名额。

去重按 id 做——HashSet<String> seen 记录已出现的 id，seen.add(rc.getId()) 返回 true 表示第一次出现，加入 dedup；返回 false 表示已存在，跳过。保留第一个出现的，后续重复的丢弃。

为什么按 id 去重而不是按 text？

不同文档中可能碰巧有相同的文本片段（比如通用的免责声明），它们是不同的 Chunk，有不同的 id，不应该被去重。按 id 去重是精确去重——只过滤真正的重复条目（同一个 Chunk 被多条检索通路命中），不误杀内容相同但来源不同的 Chunk。

5. 短路优化

```
if (topN <= 0 || dedup.size() <= topN) {
    return dedup;
}
```

去重后如果候选数不超过 topN，没必要调 Rerank API——本来就不需要裁剪，全部返回就行。这种情况在候选较少或 topN 设得较大时会触发，省去一次 HTTP 调用的开销。

6. 百炼 Rerank API 协议

百炼的 Rerank 接口是非 OpenAI 标准的格式。看一个完整的请求和响应。

请求 (POST <https://dashscope.aliyuncs.com/api/v1/services/rerank/text-rerank/text-reranker>):

```
{
  "model": "gte-rerank",
  "input": {
    "query": "AirPods Pro 2 的保修期是多久? ",
    "documents": [
      "AirPods Pro 2 提供一年有限保修...",
      "AirPods Pro 2 的电池容量为...",
      "AirPods Pro 2 支持主动降噪..."
    ]
  },
  "parameters": {
    "top_n": 2,
    "return_documents": true
  }
}
```

请求体结构：model 指定 Rerank 模型名，input 包含 query（用户问题）和 documents（候选文本数组），parameters 包含 top_n（返回条数）和 return_documents（是否在响应中返回原文）。

documents 数组从 candidates 的 text 字段提取。如果 text 为 null，替换为空字符串——防止 JSON 中出现 null 值导致 API 报错。

响应：

```
{
  "output": {
    "results": [
      {
        "index": 0,
        "relevance_score": 0.95,
        "document": {"text": "AirPods Pro 2 提供一年有限保修..."}
      },
      {
        "index": 2,
        "relevance_score": 0.31,
        "document": {"text": "AirPods Pro 2 支持主动降噪..."}
      }
    ]
  },
  "usage": {
    "total_tokens": 156
  }
}
```

output.results 是排好序的结果数组（按 relevance_score 降序）。每个结果包含 index（对应请求中 documents 数组的下标）和 relevance_score（相关度分数，0~1）。

注意 index 的关键作用——它把 Rerank 结果映射回原始的 candidates 列表。Rerank API 只接收纯文本，不知道 Chunk 的 id。通过 index 找到原始 Chunk，才能把 id、text、relevance_score 三个信息关联起来。

7. 响应解析与分数映射

doRerank 中解析响应的逻辑：

```
for (JsonElement elem : results) {
  if (!elem.isJsonObject()) continue;
  JsonObject item = elem.getAsJsonObject();

  if (!item.has("index")) continue;
  int idx = item.get("index").getAsInt();

  if (idx < 0 || idx >= candidates.size()) continue;

  RetrievedChunk src = candidates.get(idx);
```

```
Float score = null;
if (item.has("relevance_score") && !item.get("relevance_score").isJsonNull()) {
    score = item.get("relevance_score").getAsFloat();
}

RetrievedChunk hit = score != null
    ? new RetrievedChunk(src.getId(), src.getText(), score) : src;
reranked.add(hit);
addedIds.add(src.getId());

if (reranked.size() >= topN) break;
}
```

逐条处理 results 中的每个结果：

- 1. 通过 index 找到原始 candidates 列表中对应该位置的 Chunk——candidates.get(idx)
- 2. 提取 relevance_score，如果有的话，用新分数构建新的 RetrievedChunk（保留原始 id 和 text，替换 score）
- 3. 记录已添加的 Chunk id 到 addedIds Set
- 4. 达到 topN 后提前 break

防御性检查很充分：index 越界跳过、缺少字段跳过、relevance_score 为 null 时保留原始 Chunk。这些检查处理了 API 返回非预期数据的情况。

8. 回填逻辑

响应解析之后，如果 reranked 的数量不够 topN，还有一段回填逻辑：

```
if (reranked.size() < topN) {
    for (RetrievedChunk c : candidates) {
        if (addedIds.add(c.getId())) {
            reranked.add(c);
        }
        if (reranked.size() >= topN) {
            break;
        }
    }
}
```

什么时候会触发回填？Rerank API 返回的结果数可能少于请求的 topN——比如模型认为大部分候选和 query 完全无关，主动过滤掉了低分结果；或者 API 实现有其他限制。

回填策略：从原始 candidates 列表中按顺序遍历，用 addedIds.add(c.getId()) 检查是否已被 Rerank 选中——Set.add() 返回 true 表示之前没有，可以补入；返回 false 表示已在结果中，跳过。补齐到 topN 条后停止。

回填的 Chunk 保留的是向量检索阶段的原始 score（不是 Rerank 分数），质量不如 Rerank 精排选出的。但保证返回给大模型的 Chunk 数量达到 topN——有总比没有好。

9. 完整走查示例

用一个具体例子把去重、调 API、解析、回填串起来走一遍。

假设混合检索返回 8 个 Chunk（B 被向量检索和 BM25 同时命中，出现两次）：

原始位置	Chunk ID	文本摘要	来源
0	A	AirPods Pro 2 提供一年有限保修...	向量
1	B	AppleCare+ 可延长至两年保修...	向量
2	C	AirPods Pro 2 的电池容量为...	BM25
3	D	AirPods Pro 2 支持主动降噪...	BM25

4	E	AirPods Pro 2 的蓝牙版本为...	向量
5	F	AirPods Pro 2 售价 1899 元...	BM25
6	G	AirPods 3 提供一年有限保修...	向量
7	B	AppleCare+ 可延长至两年保修...	BM25（重复）

请求 topN = 5。

第一步：去重

seen Set 按 id 去重。位置 7 的 B 和位置 1 的 B id 相同，被过滤。

去重后 7 个：[A, B, C, D, E, F, G]

第二步：短路检查

dedup.size() = 7 > topN = 5，不满足短路条件，进入 doRerank。

第三步：调百炼 Rerank API

发送 documents: ["A的文本", "B的文本", "C的文本", "D的文本", "E的文本", "F的文本", "G的文本"], top_n: 5。

假设百炼返回 4 条结果（G 和 E 被模型判定为低相关，过滤了）：

```
{
  "output": {
    "results": [
      {"index": 0, "relevance_score": 0.95},
      {"index": 1, "relevance_score": 0.88},
      {"index": 6, "relevance_score": 0.42},
      {"index": 3, "relevance_score": 0.38}
    ]
  }
}
```

第四步：响应解析

API 结果	index	映射到的 Chunk	relevance_score	加入 reranked
结果 1	0	A	0.95	reranked[0] = A(0.95)
结果 2	1	B	0.88	reranked[1] = B(0.88)
结果 3	6	G	0.42	reranked[2] = G(0.42)
结果 4	3	D	0.38	reranked[3] = D(0.38)

reranked 现在有 4 条，addedIds = {A, B, G, D}。

第五步：回填

reranked.size() = 4 < topN = 5，触发回填。

从 candidates = [A, B, C, D, E, F, G] 中遍历：

- A → addedIds 已有 → 跳过
- B → addedIds 已有 → 跳过
- C → addedIds 没有 → 加入！reranked[4] = C（保留原始向量检索分数）
- reranked.size() = 5 == topN → break

最终结果：[A(0.95), B(0.88), G(0.42), D(0.38), C(原始分数)]——5 条，满足 topN。

前 4 条是 Rerank 精排的结果，第 5 条（C）是从原始候选中回填的。C 没有经过精排，但它是向量检索认为相关的候选（否则不会出现在候选列表中），作为补充是合理的。

NoopRerankClient：空对象模式

1. 完整代码

```
@Service
public class NoopRerankClient implements RerankClient {
```



```
@Override
public String provider() {
    return ModelProvider.NOOP.getId();
}

@Override
public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates,
                                   int topN, ModelTarget target) {
    if (candidates == null || candidates.isEmpty()) {
        return List.of();
    }
    if (topN <= 0 || candidates.size() <= topN) {
        return candidates;
    }
    return candidates.stream()
        .limit(topN)
        .collect(Collectors.toList());
}
```

整个类 48 行，rerank 方法的逻辑极其简单：候选数超过 topN 就截取前 topN 条，否则原样返回。不发任何 HTTP 请求，不调任何 Rerank API，不做任何相关度计算。

2. 设计意图

不是所有部署环境都有 Rerank 模型可用。Rerank 是公有云 API 服务（百炼的 gte-rerank），需要付费。如果用户没有买百炼的 Rerank 服务，或者在本地开发环境（只有 Ollama）跑项目，没有 Rerank 模型。这时候系统应该怎么办？如果直接报错“没有可用的 Rerank 模型”，用户连问答功能都用不了——而 Rerank 只是检索阶段的一个优化环节，不是核心必需。没有 Rerank，用粗排（向量检索）的结果截断一下也能用，只是精度差一些。NoopRerankClient 就是这个兜底：它实现了 RerankClient 接口，注册为 Spring Bean，但什么实际工作都不做。在 YAML 配置中作为最低优先级的候选：

```
rerank:
  default-model: bailian-rerank
  candidates:
    - id: bailian-rerank
      provider: bailian
      model: gte-rerank
      priority: 1          # 高优先级
    - id: rerank-noop
      provider: noop
      model: noop
      priority: 100       # 最低优先级，兜底
```

正常情况下，ModelSelector 排序后百炼排第一，noop 排最后。executeWithFallback 先尝试百炼——成功就返回精排结果。如果百炼失败（API 报错、熔断），markFailure 后切换到下一个候选 noop——简单截断返回。

3. 降级场景走查

走查一个降级场景：

1. 候选列表：[bailian-rerank(priority=1), rerank-noop(priority=100)]
2. 用户提问，RoutingRerankService.rerank() 调用 executeWithFallback
3. 尝试 bailian-rerank → 百炼 API 返回 HTTP 429（请求频率超限）→ catch → markFailure("bailian-rerank")
4. 尝试 rerank-noop → NoopRerankClient.rerank() → 截取前 topN 条 → 成功返回 → markSuccess("rerank-noop")
5. 用户拿到的结果没有经过精排，相关度排序可能不如有 Rerank 时准确，但至少有效果——系统没有报错
6. 后续请求：如果百炼持续不可用，连续失败达到熔断阈值（默认 2 次），bailian-rerank 被熔断。ModelSelector 在选择阶段就把它排除，直接走 noop，不再浪费时间尝试百炼
7. 30 秒冷却期过后，bailian-rerank 进入 HALF_OPEN 状态，允许一个探测请求——如果百炼恢复了，探测成功，回到 CLOSED，后续请求重新走百炼精排

这就是空对象模式 + 路由容错的协同效果：系统不会因为一个非核心组件（Rerank）的故障而整体不可用。从有 Rerank 精排到无 Rerank 截断，对用户来说只是回答质量略降（排序不那么精准），而不是功能不可用。

noop 供应商的特殊处理

回顾第二篇的内容：ModelSelector.buildModelTarget() 中，如果供应商是 noop，允许 ProviderConfig 为 null——因为 NoopRerankClient 不需要 URL、API Key 这些配置，ModelTarget 的 provider 字段为 null 是合法的。

ModelProvider.NOOP.matches(candidate.getProvider()) 做了这个特殊判断。

RoutingRerankService：路由服务

1. 完整代码

```
@Service
@Primary
public class RoutingRerankService implements RerankService {

    private final ModelSelector selector;
    private final ModelRoutingExecutor executor;
    private final Map<String, RerankClient> clientsByProvider;

    public RoutingRerankService(ModelSelector selector,
                               ModelRoutingExecutor executor,
                               List<RerankClient> clients) {
        this.selector = selector;
        this.executor = executor;
        this.clientsByProvider = clients.stream()
            .collect(Collectors.toMap(RerankClient::provider, Function.identity()));
    }

    @Override
    public List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates, int topN) {
        return executor.executeWithFallback(
            ModelCapability.RERANK,
            selector.selectRerankCandidates(),
            target -> clientsByProvider.get(target.candidate().getProvider()),
            (client, target) -> client.rerank(query, candidates, topN, target)
        );
    }
}
```

2. 三种能力路由的统一收束

把三种能力的路由代码放在一起看：

```
// Chat（同步）
executor.executeWithFallback(
    ModelCapability.CHAT,
    selector.selectChatCandidates(thinking),
    target -> clientsByProvider.get(target.candidate().getProvider()),
    (client, target) -> client.chat(request, target)
);

// Embedding
executor.executeWithFallback(
    ModelCapability.EMBEDDING,
    selector.selectEmbeddingCandidates(),
    this::resolveClient,
    (client, target) -> client.embed(text, target)
);

// Rerank
executor.executeWithFallback(
```

```
ModelCapability.RERANK,
selector.selectRerankCandidates(),
target -> clientsByProvider.get(target.candidate().getProvider()),
(client, target) -> client.rerank(query, candidates, topN, target)
);
```

三者结构一模一样——四个参数（能力类型、候选列表、客户端解析、调用函数）——对应。executeWithFallback 的泛型设计在这里完美收束：

能力	C (客户端类型)	T (返回类型)
Chat	ChatClient	String
Embedding	EmbeddingClient	List<Float>
Rerank	RerankClient	List<RetrievedChunk>

同一套路由机制（ModelSelector 选候选 → ModelRoutingExecutor 遍历尝试 → ModelHealthStore 熔断保护），三种能力共用。加新能力时只需要：定义接口 → 实现客户端 → 加路由服务 → 加配置，路由和熔断的代码不用改一行。这就是第一篇讲的三种能力并行结构在代码层面的最终落地。

Token 估算：HeuristicTokenCounterService

1. 接口定义

```
public interface TokenCounterService {

    /**
     * 统计文本的 Token 数
     *
     * @param text 文本内容
     * @return Token 数（无法计算时返回 null）
     */
    Integer countTokens(String text);
}
```

2. 完整代码

```
@Service
public class HeuristicTokenCounterService implements TokenCounterService {

    @Override
    public Integer countTokens(String text) {
        if (!StringUtils.hasText(text)) {
            return 0;
        }

        int asciiCount = 0;
        int cjkCount = 0;
        int otherCount = 0;

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (Character.isWhitespace(ch)) {
                continue;
            }
            if (ch <= 0x7F) {
                asciiCount++;
            } else if (isCjk(ch)) {
                cjkCount++;
            } else {
            }
```

```
        otherCount++;
    }
}

int asciiTokens = (asciiCount + 3) / 4;
int otherTokens = (otherCount + 1) / 2;
int total = asciiTokens + cjkCount + otherTokens;
return Math.max(total, 1);
}

private boolean isCjk(char ch) {
    Character.UnicodeBlock block = Character.UnicodeBlock.of(ch);
    return block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_A
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_B
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_C
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_D
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_E
        || block == Character.UnicodeBlock.CJK_UNIFIED_IDEOGRAPHS_EXTENSION_F
        || block == Character.UnicodeBlock.CJK_COMPATIBILITY_IDEOGRAPHS
        || block == Character.UnicodeBlock.CJK_COMPATIBILITY_IDEOGRAPHS_SUPPLEMENT
        || block == Character.UnicodeBlock.CJK_RADICALS_SUPPLEMENT
        || block == Character.UnicodeBlock.CJK_SYMBOLS_AND_PUNCTUATION
        || block == Character.UnicodeBlock.HIRAGANA
        || block == Character.UnicodeBlock.KATAKANA
        || block == Character.UnicodeBlock.KATAKANA_PHONETIC_EXTENSIONS
        || block == Character.UnicodeBlock.HANGUL_SYLLABLES
        || block == Character.UnicodeBlock.HANGUL_JAMO
        || block == Character.UnicodeBlock.HANGUL_COMPATIBILITY_JAMO;
}
}
```

3. 三种字符分类的估算规则

算法的核心思路：按字符类型分类计数，对每类应用不同的估算系数。

字符类型	判断条件	估算规则	公式
ASCII	ch <= 0x7F	约 4 个字符 ≈ 1 Token	(asciiCount + 3) / 4 (向上取整)
CJK	isCjk(ch)	约 1 个字符 ≈ 1 Token	cjkCount
其他	以上都不是	约 2 个字符 ≈ 1 Token	(otherCount + 1) / 2 (向上取整)
空白	Character.isWhitespace(ch)	跳过不计	—

这些系数从哪来？大多数主流 Tokenizer (tiktoken、sentencepiece) 的经验值：

- 英文的 subword 分词粒度通常在 3~5 个字符一个 Token，取 4 是中间值
- 中文每个汉字通常独立成 Token 或两个字组合一个 Token，取 1 字 ≈ 1 Token 偏保守（实际可能略多）
- 日文假名、韩文音节等也归入 CJK，处理方式相同

isCjk 方法覆盖了中日韩统一表意文字（含多个扩展区）、兼容表意文字、部首补充、CJK 符号和标点、平假名、片假名、韩文音节等 Unicode 块。覆盖面很全，能正确识别绝大多数中日韩文字。

4. 走查示例

假设文本是 "Hello 你好世界! This is a test":

逐字符分类：

字符	类型	说明
H, e, l, l, o	ASCII	英文字母

你, 好, 世, 界	空白	跳过
	CJK	中文汉字
!	其他	全角感叹号 (U+FF01), 属于 HALFWIDTH_AND_FULLWIDTH_FORMS , 不在 isCjk 覆盖范围内
T, h, i, s	空白	跳过
	ASCII	
i, s	空白	跳过
	ASCII	
	空白	跳过
a	ASCII	
	空白	跳过
t, e, s, t	空白	跳过
	ASCII	

统计结果: asciiCount = 16, cjkCount = 4, otherCount = 1
计算:

```
asciiTokens = (16 + 3) / 4 = 4
otherTokens = (1 + 1) / 2 = 1
total = 4 + 4 + 1 = 9
```

实际用 tiktoken (cl100k_base 编码) 计算这段文本大约 11~12 Token。启发式估算 9 Token，误差约 20%。对于 Token 预算控制（比如判断对话历史是否超过 2000 Token 的预算线）来说，这个误差完全可接受。

5. 为什么不用精确 Tokenizer

精确的 Token 计算需要加载模型的词表文件。比如用 tiktoken-java 库计算 GPT 系列模型的 Token 数、用 sentencepiece 计算 Qwen 系列的 Token 数。
几个问题：

- 额外依赖：**引入 tiktoken-java 或 sentencepiece 库，增加 jar 包体积和潜在的兼容性问题
- 模型绑定：**不同模型的 Tokenizer 不同，GPT 用 cl100k_base，Qwen 用自己的 BPE 词表。系统支持多个模型供应商，精确计算需要按模型切换 Tokenizer
- 初始化开销：**词表文件加载需要时间和内存，尤其是大词表
- 使用场景不需要精确：**Token 估算的主要用途是会话记忆管理——估算对话历史占用多少 Token，决定保留多少轮历史。预算线本身就是一个软限制（设 2000 Token 和设 2200 Token 差别不大），±20% 的误差不影响决策

启发式估算零依赖、零延迟、跨模型通用。如果未来某个场景确实需要精确计算（比如按 Token 计费的成本统计），替换 TokenCounterService 的实现即可——接口已经定义好了，调用方不需要改。

响应清洗：LLMResponseCleaner

1. 完整代码

```
@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public final class LLMResponseCleaner {

    private static final Pattern LEADING_CODE_FENCE =
        Pattern.compile("^```[\\w-]*\\s*\\n?");
    private static final Pattern TRAILING_CODE_FENCE =
        Pattern.compile("\\n?```\\s*$");

    /**
     * 移除 Markdown 代码块围栏（例如 ```json ... ```）
     */
    public static String stripMarkdownCodeFence(String raw) {
```

```
        if (raw == null) {
            return null;
        }
        String cleaned = raw.trim();
        cleaned = LEADING_CODE_FENCE.matcher(cleaned).replaceFirst("");
        cleaned = TRAILING_CODE_FENCE.matcher(cleaned).replaceFirst("");
        return cleaned.trim();
    }
}
```

2. 使用场景

在 RAG 系统中，有些环节会让大模型输出结构化数据。比如意图识别——Prompt 里要求模型判断用户的意图，以 JSON 格式输出：

请根据用户的消息判断意图类型，以 JSON 格式输出，包含 intent 和 confidence 字段。

模型的回答通常是：

```
{"intent": "knowledge_retrieval", "confidence": 0.95}
```

但有些模型会自作主张地包一层 Markdown 代码围栏：

```
```json
{"intent": "knowledge_retrieval", "confidence": 0.95}
```
```

直接拿这段文本做 gson.fromJson(response, ...) 会报 JSON 解析异常——因为开头有 ```json\n，结尾有 \n```，这不是合法的 JSON。

stripMarkdownCodeFence 用两个正则把开头和结尾的围栏去掉：

处理前：

```
```json
{"intent": "knowledge_retrieval", "confidence": 0.95}
```
```

处理后：

```
{"intent": "knowledge_retrieval", "confidence": 0.95}
```

两个正则的设计：

```[\w-]\*\s\*\n?——匹配开头的围栏。^ 锚定行首，``` 匹配三个反引号，[\w-]\* 匹配可选的语言标识（json、xml、markdown、c++ 中的连字符等），\s\*\n? 匹配尾部空白和可选换行

\n?```\s\*\$——匹配结尾的围栏。\\n? 匹配可选的前导换行，``` 匹配三个反引号，\\s\*\$ 匹配尾部空白并锚定行尾

两个 Pattern 对象是类级别的静态常量——Pattern.compile 在类加载时执行一次，后续每次调用 stripMarkdownCodeFence 直接用编译好的正则，避免重复编译的开销。这个方法可能被高频调用（每次大模型返回 JSON 格式的响应都需要清洗），预编译是必要的。

3. 防御性设计

stripMarkdownCodeFence 对没有围栏的文本也安全——正则匹配不到就不替换，原文不变。所以可以无脑调用：不管模型有没有加围栏，都过一遍清洗，有则去之，无则不动。

系列总结

到这里，整个大模型调度引擎实战系列八篇文章就写完了。infra-ai 模块的 9 个包、约 40 个类全部讲完。回顾一下完整的知识脉络：

**第一篇《AI 基础设施层宏观设计》**——建立全局视角。为什么需要独立的 AI 基础设施层，infra-ai 模块的 9 个包职责划分，三层接口设计（业务层 → 路由层 → 供应层），配置驱动的设计理念，六种核心设计模式。

**第二篇《多模型路由与智能选择》**——路由入口。ModelSelector 的选择算法：优先级排序、首选模型提升、深度思考过滤、ModelTarget 构建、ModelUrlResolver URL 解析、熔断检查两层机制。

**第三篇《三态熔断器与故障转移》**——容错骨架。ModelHealthStore 三态状态机（CLOSED → OPEN → HALF\_OPEN），ConcurrentHashMap.compute() 并发安全，ModelRoutingExecutor.executeWithFallback() 通用同步故障转移执行器，ModelCaller<C, T> 泛型设计。

**第四篇《Chat 同步调用与模板方法》**——协议封装。AbstractOpenAIStyleChatClient 模板方法模式，doChat 同步调用九步骤，三个钩子方法（requiresApiKey、customizeRequestBody、isReasoningEnabledForStream），HTTP 错误处理体系（ModelClientErrorType 分类 + ModelClientException 结构化异常）。

**第五篇《SSE 流式解析与异步执行》**——流式底层。doStreamChat 模板方法，OpenAIStyleSseParser SSE 行解析（data: 前缀、[DONE] 标记、delta.content / delta.reasoning\_content），StreamAsyncExecutor 异步提交到专用线程池，StreamCancellationHandle 双重取消机制（AtomicBoolean 信号 + Call.cancel() 中断 I/O）。

**第六篇《流式路由的首包探测机制》**——流式路由。RoutingLLMService.streamChat() 的 probe-and-commit 模式：ProbeBufferingCallback 探测缓冲装饰器（synchronized 保护两阶段切换），FirstPacketAwaiter 首包等待器（CountDownLatch 跨线程同步），四种探测结果（SUCCESS / ERROR / TIMEOUT / NO\_CONTENT），在异步回调场景下实现对前端透明的故障转移。

**第七篇《Embedding 向量化客户端》**——向量化子系统。OllamaEmbeddingClient (/api/embed 端点、dimensions 参数) 和 SiliconFlowEmbeddingClient (/v1/embeddings 端点、批量分片 max 32、encoding\_format: "float") 的 API 协议差异，为什么不用模板方法基类，RoutingEmbeddingService 复用 executeWithFallback。

**第八篇《Rerank 重排序与辅助工具》**——收尾。BaiLianRerankClient 百炼重排序（去重 + API 调用 + 回填），NoopRerankClient 空对象模式优雅降级，RoutingRerankService 三种能力路由的统一收束，HeuristicTokenCounterService 启发式 Token 估算，LLMResponseCleaner Markdown 代码围栏清洗。

八篇文章从宏观到微观、从核心到辅助，逐层深入。贯穿始终的核心设计理念：

- 三层接口隔离关注点——业务层只看到 LLMService / EmbeddingService / RerankService，不感知供应商和路由细节
- 配置驱动零代码切换——供应商、模型、优先级、熔断参数全部在 YAML 中配置，切换供应商不改一行代码
- 一套路由机制三种能力共用——ModelSelector + ModelRoutingExecutor + ModelHealthStore 是通用骨架，Chat / Embedding / Rerank 插入不同的 ModelCaller 实现即可
- 设计模式让扩展只加不改——策略模式（供应商可插拔）、模板方法（协议复用）、装饰器（探测缓冲）、空对象（安全降级）、注册表（自动发现）

这些组件共同构成了 Ragent 项目的 AI 基础设施层。业务层的 RAG 检索、问答、文档入库、意图识别等功能，都建立在这个基础设施之上。